

Machine Learning Foundations

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Emmanuel Djegou, PhD

emmanueldjegou5@gmail.com

Module 1 — Learning from Data

1. What is a learning algorithm?
2. Tasks: classification, regression, and more
3. Performance measures
4. Supervised vs. unsupervised learning
5. Example: linear regression
6. Capacity, overfitting, underfitting
7. Regularization
8. Hyperparameters and validation sets
9. Cross-validation

Module 2 — Statistical Foundations

10. Point estimation
11. Bias of an estimator
12. Variance and standard error
13. Bias-variance tradeoff and MSE
14. Consistency
15. Maximum likelihood estimation
16. MLE: linear regression connection
17. Properties of MLE

What Is a Learning Algorithm?

Mitchell's definition (1997)

*"A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."*

The three ingredients:

- ▶ T — **Task**: what we want the system to do (e.g. classify images, predict prices)
- ▶ P — **Performance**: a quantitative measure of how well it does T
- ▶ E — **Experience**: the data the system learns from

Key insight:

Learning is *not* the task itself. Learning is the means of attaining the ability to perform the task.

Example: teaching a robot to walk. **Walking** is the task. The robot **learns** to walk through experience (practice, feedback, data).

The Task T : What Machines Can Learn to Do

An **example** is a vector $\mathbf{x} \in \mathbb{R}^n$ of measured features. Common tasks include:

- ▶ **Classification:** $f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$. Assign an input to one of k categories.
Example: object recognition, face detection.
- ▶ **Regression:** $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Predict a numerical value.
Example: insurance claim amounts, stock prices.
- ▶ **Transcription:** Convert unstructured data to discrete text.
Example: OCR, speech recognition.
- ▶ **Machine translation:** Convert a sequence in one language to another.
- ▶ **Anomaly detection:** Flag unusual events.
Example: credit card fraud detection.
- ▶ **Synthesis & sampling:** Generate new examples similar to training data.
Example: texture generation, speech synthesis.
- ▶ **Denoising:** Recover a clean example $\mathbf{x}$ from a corrupted version $\tilde{\mathbf{x}}$.
- ▶ **Density estimation:** Learn $p_{\text{model}}(\mathbf{x})$, a probability distribution over the data space.

The Performance Measure P

P must be **quantitative**, **task-specific**, and measured on **held-out test data** (not training data).

Common choices:

- ▶ **Accuracy / error rate:** fraction correct or incorrect. Used for classification and transcription.
- ▶ **0-1 loss:** 0 if correct, 1 if not. Expected 0-1 loss = error rate.
- ▶ **Mean squared error (MSE):** for regression tasks.
- ▶ **Average log-probability:** for density estimation.

Why the test set?

We care about performance on **new, unseen inputs** — not on the data the model was trained on. The test set estimates performance in deployment.

Choosing P is harder than it seems:

- ▶ For transcription: accuracy per sequence, or per character?
- ▶ For regression: penalise frequent medium errors, or rare large ones?
- ▶ Some ideal measures are *computationally intractable*.

The Experience E : Supervised vs. Unsupervised Learning

Supervised learning

Each example $\mathbf{x}$ is paired with a **label or target** y .

Goal: learn to predict y from $\mathbf{x}$, i.e. estimate $p(y \mid \mathbf{x})$.

Examples: classification, regression, structured output.

Think of y as provided by a *teacher* who shows the system what to do.

Unsupervised learning

Examples $\mathbf{x}$ only — **no labels**.

Goal: learn the structure of the data, i.e. estimate $p(\mathbf{x})$ or discover useful properties.

Examples: clustering, density estimation, denoising, synthesis.

No teacher — the algorithm must make sense of the data on its own.

The boundary is blurry. The chain rule lets us decompose $p(\mathbf{x})$ into n supervised problems, and supervised learning can be reduced to estimating the joint $p(\mathbf{x}, y)$ via unsupervised methods.

Beyond Supervised and Unsupervised

Semi-supervised learning

Some examples have labels; others do not. The algorithm must use both the labelled and unlabelled portions.

Multi-instance learning

A *collection* of examples is labelled as a whole (contains/doesn't contain a class), but individual members are not labelled.

Reinforcement learning

The algorithm *interacts with an environment*. There is a feedback loop: actions produce rewards or penalties. The algorithm learns a policy by trial and error.

(Beyond the scope of these modules; see Sutton & Barto, 1998.)

The design matrix

Most algorithms work on a design matrix $X \in \mathbb{R}^{m \times n}$: m examples (rows), n features (columns). For supervised learning, a label vector $\mathbf{y} \in \mathbb{R}^m$ accompanies X .

Example: Linear Regression

The simplest complete learning algorithm. Task: predict scalar $y \in \mathbb{R}$ from $\mathbf{x} \in \mathbb{R}^n$.

Model

$$\hat{y} = \mathbf{w}^\top \mathbf{x} \quad (\text{or with bias: } \hat{y} = \mathbf{w}^\top \mathbf{x} + b)$$

$\mathbf{w} \in \mathbb{R}^n$ are the **parameters** (weights). Each w_i controls how feature x_i affects the prediction.

Performance measure: Mean Squared Error (MSE)

$$\text{MSE}_{\text{test}} = \frac{1}{m} \sum_i \left(\hat{y}_i^{(\text{test})} - y_i^{(\text{test})} \right)^2$$

Training: minimise $\text{MSE}_{\text{train}}$

Setting $\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$ gives the **normal equations**:

$$\mathbf{w} = \left(\mathbf{X}^{(\text{train})\top} \mathbf{X}^{(\text{train})} \right)^{-1} \mathbf{X}^{(\text{train})\top} \mathbf{y}^{(\text{train})}$$

Linear regression is T, P, E all in one: the task is regression, the performance is MSE, the experience is the training set of (x, y) pairs.

Generalization: The Central Challenge

Key distinction

Training error: error on the data used to fit the model.

Generalization error (test error): expected error on *new, unseen* inputs drawn from the same distribution.

The i.i.d. assumption:

- ▶ Training and test examples are **independent**
- ▶ Both are drawn from the same **data generating distribution** p_{data}

This allows us to reason about the relationship between training and test error mathematically.

The fundamental tension:

- ① Make training error small
- ② Keep test error close to training error

Underfitting vs. Overfitting

- ▶ **Underfitting:** model fails to achieve low training error. The model is too simple.
- ▶ **Overfitting:** training error is low but test error is much higher. The model memorized the training data.

Bayes error: the irreducible error of the true distribution — the best any model can achieve.

Model Capacity

Capacity

A model's **capacity** is its ability to fit a wide variety of functions — its expressiveness. It is controlled by the **hypothesis space**: the set of functions the algorithm can choose from.

Polynomial regression example:

Degree 1 (linear): $\hat{y} = b + w_1x$

Degree 2 (quadratic): $\hat{y} = b + w_1x + w_2x^2$

Degree 9: $\hat{y} = b + \sum_{i=1}^9 w_ix^i$

Higher degree $\Rightarrow$ larger hypothesis space $\Rightarrow$ higher capacity.

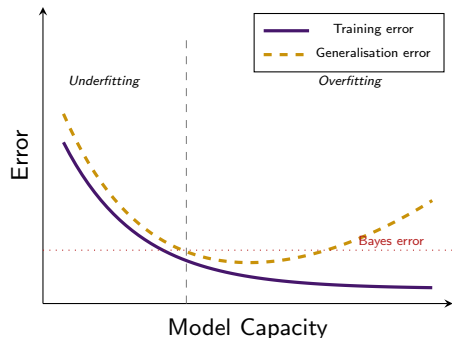
The optimal capacity rule:

- ▶ Too low: *underfits* — cannot capture the true pattern
- ▶ Just right: generalises well
- ▶ Too high: *overfits* — memorises training noise

Representational vs. effective capacity:

Representational capacity = what the model family can express. Effective capacity = what the optimiser actually achieves (may be lower due to optimisation limitations).

Capacity, Error, and the U-Shaped Curve



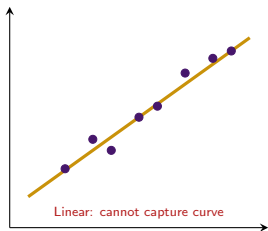
Reading the curve:

- ▶ Training error *decreases monotonically* as capacity grows
- ▶ Generalisation error has a **U-shape**: initially high (underfitting), then low (optimal), then rises again (overfitting)
- ▶ The gap between training and generalisation error is the **generalisation gap**
- ▶ More training data *shifts the optimum* to higher capacity and *lowers* the generalisation error curve

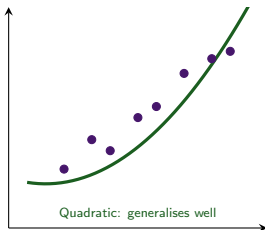
VC dimension: a formal measure of binary classifier capacity (Vapnik & Chervonenkis, 1971). The discrepancy between training and generalisation error is bounded above by a quantity that grows with capacity and shrinks with m .

Underfitting, Overfitting: A Visual

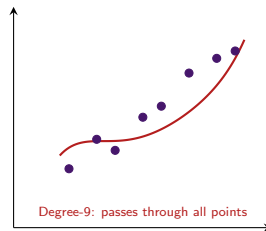
Underfitting



Good fit



Overfitting



Same training data. Same goal (fit a quadratic relationship). Three very different outcomes depending on model capacity.

The No Free Lunch Theorem

No Free Lunch (Wolpert, 1996)

Averaged over **all possible data-generating distributions**, every classification algorithm has the same expected error on unseen data.

In other words: no algorithm is universally better than any other.

What this means:

- ▶ A sophisticated algorithm and a trivial one (e.g., always predict the same class) have the same *average* error across all distributions
- ▶ There is no universally best learning algorithm
- ▶ There is no universally best form of regularization

What this does NOT mean:

We don't care about all distributions equally. We care about the distributions that arise in real-world problems.

The goal of ML research: understand what distributions appear in practice and design algorithms that perform well on those.

We must design algorithms with **inductive biases** suited to the structure of our problem domain.

Regularization: Expressing Preferences

Regularization is any modification to a learning algorithm that *reduces generalisation error without reducing training error's minimum*.

Weight decay (L2 regularization):

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

- ▶ $\lambda = 0$: no preference, standard training
- ▶ Large λ : forces weights toward zero
- ▶ Result: *smoother* functions with smaller coefficients

The regularizer $\Omega(\mathbf{w}) = \mathbf{w}^T \mathbf{w}$ is a penalty added to the cost.

Effect of λ on a degree-9 model:

- ▶ Large λ : flat function (underfits) — only constant can be learnt
- ▶ Medium λ : recovers the correct shape — regularization saves the day
- ▶ $\lambda \rightarrow 0$: degree-9 polynomial overfits wildly

General view: regularization expresses a preference for one function over another. It is a *softer* version of excluding functions from the hypothesis space (exclusion = infinitely strong preference against).

Hyperparameters and Validation Sets

Hyperparameters

Settings that **control the behaviour** of the learning algorithm but are **not learned** by the algorithm itself from the training data.

Examples:

- ▶ Polynomial degree d (capacity hyperparameter)
- ▶ Weight decay strength λ
- ▶ Learning rate in gradient descent
- ▶ Number of hidden layers / neurons

Why not learn hyperparameters on the training set?

Capacity hyperparameters (e.g. polynomial degree) would always be set to maximum, causing overfitting.

The three-way split

- 1 **Training set** ($\approx 80\%$ of labelled data): learn model parameters $\mathbf{w}$
- 2 **Validation set** ($\approx 20\%$ of labelled data): choose hyperparameters; estimates generalisation error *during* training
- 3 **Test set** (held out entirely): final unbiased estimate of generalisation error

The test set must **never** be used to make modelling decisions. Repeated use makes it stale.

k-Fold Cross-Validation

Problem: with small datasets, a fixed validation set gives a noisy estimate of generalisation error.

k-fold cross-validation

- 1 Partition data into k non-overlapping subsets (folds)
- 2 For $i = 1, \dots, k$: train on all folds except i ; evaluate on fold i
- 3 Report the **average test error** across k trials

Common choice: $k = 5$ or $k = 10$. Special case: $k = m$ (leave-one-out CV).

Fold 1	Test	Train	Train	Train	Train
Fold 2	Train	Test	Train	Train	Train
Fold 3	Train	Train	Test	Train	Train
Fold 4	Train	Train	Train	Test	Train
Fold 5	Train	Train	Train	Train	Test

- ▶ **Benefit:** every example used for both training and testing
- ▶ **Cost:** k times more computation

Definition

A **point estimator** is any function $\hat{\theta}_m = g(x^{(1)}, \dots, x^{(m)})$ of m i.i.d. data points. It provides a single “best guess” for an unknown parameter θ (or function f).

The frequentist view:

- ▶ θ is fixed but unknown
- ▶ The data $\{x^{(1)}, \dots, x^{(m)}\}$ is random (drawn from p_{data})
- ▶ Therefore $\hat{\theta}_m$ is a *random variable*

We want $\hat{\theta}_m$ to be *close* to the true θ , but “close” must be made precise.

Function estimation:

Estimating a function f (e.g. predicting y from x) is the same idea: the estimate $\hat{f}$ is a point estimator in function space.

Key properties we care about:

- 1 **Bias:** does $\hat{\theta}$ systematically over- or under-estimate θ ?
- 2 **Variance:** how spread out are different estimates from different samples?
- 3 **Consistency:** does $\hat{\theta} \rightarrow \theta$ as $m \rightarrow \infty$?

Definition

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}[\hat{\theta}_m] - \theta.$$

The expectation is taken over the training data distribution.

An estimator is **unbiased** if:

$$\mathbb{E}[\hat{\theta}_m] = \theta$$

Key idea: Bias measures systematic error of an estimator across different training samples.

Example 1: Bernoulli mean

$$\hat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\mathbb{E}[\hat{\theta}_m] = \theta$$

Unbiased

Example 2: Gaussian mean

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\mathbb{E}[\hat{\mu}_m] = \mu$$

Unbiased

Example 3: Sample variance

$$\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$$

$$\mathbb{E}[\hat{\sigma}_m^2] = \frac{m-1}{m} \sigma^2$$

Biased (downward)

Unbiased version

$$\tilde{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$$

Unbiased estimators are not always optimal — bias–variance tradeoff matters.

Variance and Standard Error (1/2)

Variance of an estimator

$\text{Var}(\hat{\theta})$ measures how much the estimate changes across different random samples of the training data.

Key scaling laws:

Standard error of the mean:

$$\text{SE}(\hat{\mu}_m) = \sqrt{\text{Var}\left(\frac{1}{m} \sum_{i=1}^m x^{(i)}\right)} = \frac{\sigma}{\sqrt{m}}$$

- ▶ More data $\Rightarrow$ lower uncertainty
- ▶ Decreases as $1/\sqrt{m}$

Bernoulli estimator variance:

$$\text{Var}(\hat{\theta}_m) = \frac{\theta(1 - \theta)}{m}$$

Variance and Standard Error (2/2)

Confidence intervals (CLT):

$$\hat{\mu}_m \pm 1.96 \text{SE}(\hat{\mu}_m)$$

- ▶ Approximate 95% confidence interval
- ▶ Valid for large m via Central Limit Theorem

In ML practice:

- ▶ Compare models using confidence intervals of test error
- ▶ Claim “A beats B” only if intervals do not overlap (rough rule)

Practical note

$\sqrt{\hat{\sigma}_m^2}$ and $\sqrt{\tilde{\sigma}_m^2}$ are both biased estimators of σ , but are widely used in practice (especially for large m).

Bias–Variance Tradeoff (MSE Decomposition)

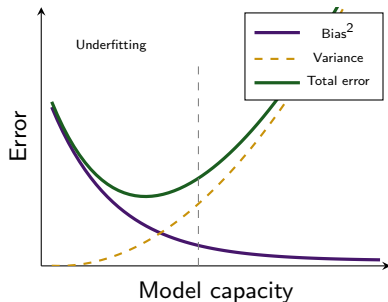
Mean Squared Error

$$\text{MSE} = \mathbb{E}[(\hat{\theta}_m - \theta)^2] = \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$

Key idea: Total error decomposes into two competing sources:

- ▶ **Bias** = systematic error
- ▶ **Variance** = sensitivity to data fluctuations

Bias–Variance Tradeoff (Intuition)



Interpretation:

- ▶ Increasing capacity **reduces bias**
- ▶ But it **increases variance**
- ▶ Total error is U-shaped

Key regimes:

- ▶ **Underfitting**: high bias, low variance
- ▶ **Overfitting**: low bias, high variance

Implication:

Regularization trades bias for lower variance

Consistency

Definition

An estimator $\hat{\theta}_m$ is **consistent** if

$$\hat{\theta}_m \xrightarrow{P} \theta \quad \text{as } m \rightarrow \infty$$

i.e. for any $\epsilon > 0$: $P(|\hat{\theta}_m - \theta| > \epsilon) \rightarrow 0$ as $m \rightarrow \infty$.

Consistent: sample mean $\hat{\mu}_m$

As m grows, $\hat{\mu}_m$ concentrates around μ .

Inconsistent (but unbiased):

$\hat{\theta} = x^{(1)}$ (just the first data point)

$\mathbb{E}[\hat{\theta}] = \theta$ for any m , so it is unbiased. But it never improves as we collect more data.

Key nuances:

- ▶ Consistency $\Rightarrow$ asymptotic unbiasedness
- ▶ Asymptotic unbiasedness does **NOT** imply consistency
- ▶ Variance of the estimator must also shrink to zero

Consistency tells us the bias diminishes as data grows. We need both: the estimator must be *right on average* and *increasingly precise*.

Maximum Likelihood Estimation

The MLE principle

Given m i.i.d. samples $X = \{x^{(1)}, \dots, x^{(m)}\}$ from unknown p_{data} , and a parametric model $p_{\text{model}}(x; \theta)$, the **maximum likelihood estimator** is:

$$\theta_{\text{ML}} = \arg \max_{\theta} \prod_{i=1}^m p_{\text{model}}(x^{(i)}; \theta)$$

Log-likelihood (numerically stable, product $\rightarrow$ sum):

$$\theta_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log p_{\text{model}}(x^{(i)}; \theta)$$

Rescaling by $1/m$ gives an *expectation* under the empirical distribution $\hat{p}_{\text{data}}$:

MLE as KL divergence minimisation:

$$D_{\text{KL}}(\hat{p}_{\text{data}} \| p_{\text{model}}) = \mathbb{E}_{\hat{p}_{\text{data}}} [\log \hat{p}_{\text{data}} - \log p_{\text{model}}]$$

Minimising KL divergence $\Leftrightarrow$ minimising $-\mathbb{E}[\log p_{\text{model}}] \Leftrightarrow$ **maximising log-likelihood**.

In software: maximise log-likelihood $\equiv$ minimise **negative log-likelihood (NLL)** $\equiv$ minimise **cross-entropy**.

MLE and Cross-Entropy: Why Loss Functions Arise Naturally

Gaussian noise model:

$$p(y | x) = \mathcal{N}(y; \hat{y}(x; \mathbf{w}), \sigma^2)$$

Log-likelihood:

$$\sum_{i=1}^m \log p(y^{(i)} | x^{(i)})$$

$$= -\frac{m}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Interpretation:

- ▶ Constant term: $-\frac{m}{2} \log(2\pi\sigma^2)$
- ▶ Data fit term: $\sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$

Conclusion: $\arg \max \log p(y|x)$
 $\equiv \arg \min \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \text{MSE}$

General principle:

Model assumption	Loss
Gaussian $p(y x)$	MSE
Bernoulli $p(y x)$	Binary cross-entropy
Categorical (softmax)	Cross-entropy

Key takeaway: every standard loss function is a *negative log-likelihood* under some probabilistic model of the output. This is a *design principle*.

MSE and cross-entropy arise **naturally** from the MLE framework. They are not arbitrary choices — they follow from Gaussian and Bernoulli/categorical distributional assumptions.

Properties of Maximum Likelihood

Why MLE is the gold standard:

- 1 **Consistency:** as $m \rightarrow \infty$, $\theta_{\text{ML}} \rightarrow \theta_{\text{true}}$ (under mild conditions)
- 2 **Statistical efficiency:** among all consistent estimators, MLE achieves the **Cramér-Rao lower bound** on MSE as $m \rightarrow \infty$ (Cramér 1946; Rao 1945)
- 3 **Asymptotically optimal:** best possible rate of convergence

Conditions for consistency of MLE:

- ▶ True p_{data} must lie within the model family $p_{\text{model}}(\cdot; \theta)$
- ▶ p_{data} must correspond to a *unique* θ

Practical caveat:

When m is small, MLE can overfit. In this regime, **regularisation** (e.g. weight decay) introduces bias to reduce variance — giving a biased but lower-MSE estimator.

Bottom line

MLE is the principled default. Regularise when data is limited.

Module Summary (1/2): Machine Learning Concepts

Module 1 takeaways:

- ▶ A learning algorithm improves at task T , measured by P , through experience E
- ▶ Supervised learning estimates $p(y|\mathbf{x})$; unsupervised learning estimates $p(\mathbf{x})$
- ▶ Generalisation = performance on unseen data (core goal of ML)
- ▶ Underfitting = model too simple; overfitting = model too complex
- ▶ Model capacity controls the bias–variance tradeoff
- ▶ No Free Lunch: no universally best algorithm; inductive bias must match the problem
- ▶ Regularisation reduces overfitting by penalising complexity
- ▶ Hyperparameters require validation sets (cross-validation when data is scarce)

Module Summary (2/2): Estimation and Learning Theory

Module 2 takeaways:

- ▶ Point estimators are functions of data; frequentist view treats θ as fixed
- ▶ Bias = systematic error; variance = sampling variability
- ▶ **Unbiased** $\neq$ **optimal**: lower-variance estimators may perform better
- ▶ $\text{MSE} = \text{Bias}^2 + \text{Variance}$ decomposes total error
- ▶ Consistency: $\hat{\theta}_m \xrightarrow{P} \theta$ as $m \rightarrow \infty$
- ▶ Maximum likelihood: maximise $\sum \log p_{\theta}(x)$
- ▶ MLE is equivalent to minimising KL divergence to the data distribution
- ▶ MSE corresponds to Gaussian noise; cross-entropy corresponds to Bernoulli/categorical models
- ▶ MLE is consistent and asymptotically efficient (Cramér–Rao bound)

Questions?

Contact Information

Author Emmanuel Djegou, PhD

Topic Area Data Science, Statistical Modeling & Survival Analysis

Email emmanueldjegou5@gmail.com

LinkedIn [linkedin.com/in/emmanuel-djegou-phd-5652b2254](https://www.linkedin.com/in/emmanuel-djegou-phd-5652b2254)